

Achieve[®]

**Writing
PAD**

No.33

Keshav
Digicode
Shree
Testing



SHEETS 60

M.R.P. ₹ 40/-
(incl. of all taxes)



Shree Parasnath Enterprises
Noida U.P.

Software Testing - Grade Mathematics

S/W Eng (V-1)

7-04-26

Tuesday

- 1) Intro → Scratch to entrepreneurship.
- 2) SDHC.
- 3) Requirement analysis (SRS).
- 4) Software Project Management (COCOMO).
- 5) Software Design (coupling, cohesion, UML).
- 6) Coding & Testing (DFD, class program).
- 7) Maintenance.
- 8) Quality Mgmt, Reuse.

—————X—————X—————X—————X—————
(V-2) S/W Eng. & Evolution

→ Systematic, cost-effective, disciplined technique for SW development.

→ Eng. approach to develop SW.

→ 1945-65 (origin)

→ 1965-85 (crisis) 100 → 2 useful

→ 1990-2000 (internet) → 50 useless
Failure :- OS/360

→ 2000-2010 (lightweight)

→ 2010 - Till (AI, ML, DL)

—————X—————X—————X—————
PTO →

Waterfall (classical)

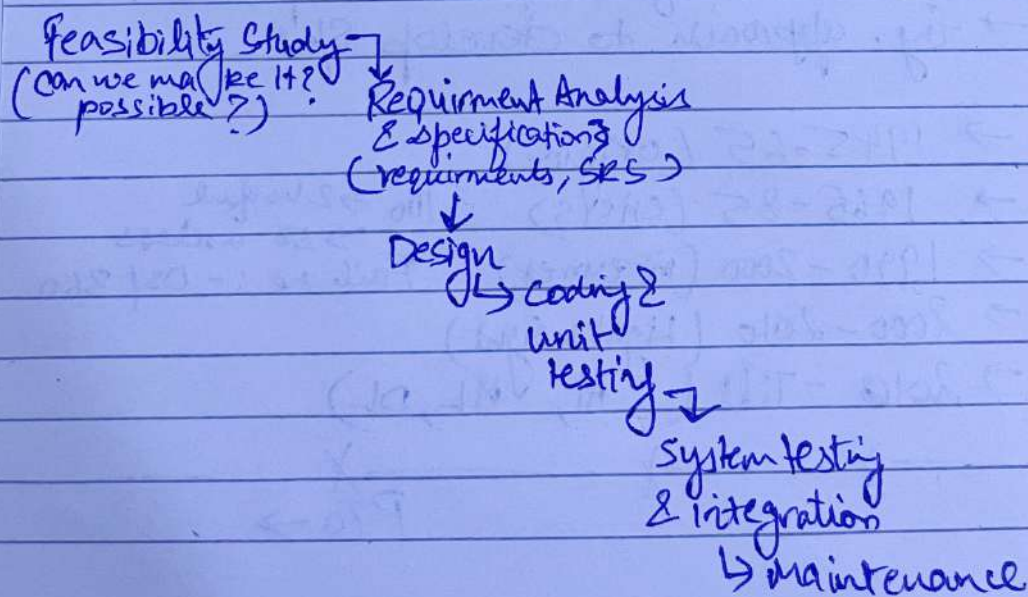
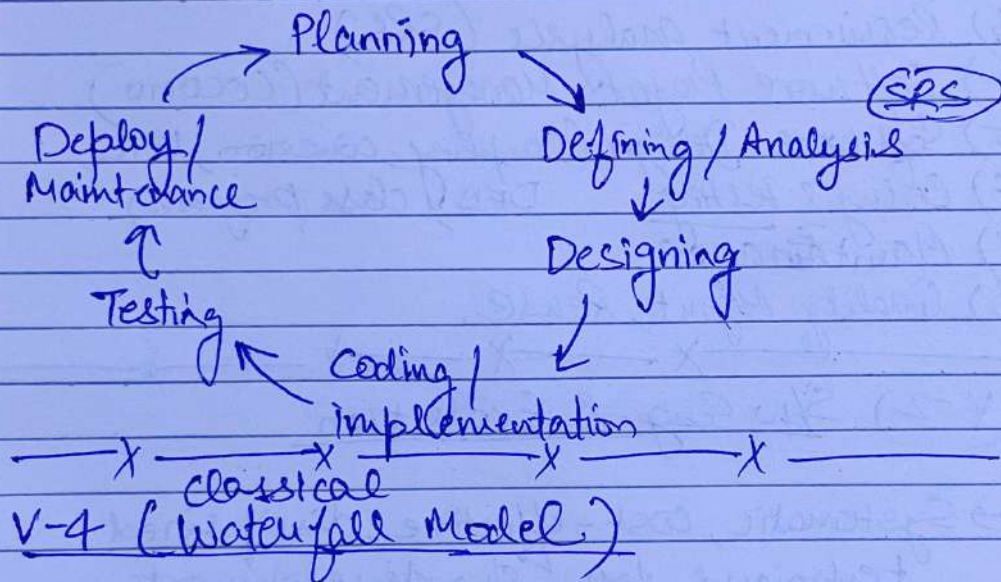
Advantages

- 1) Basic model
- 2) Simple & easy
- 3) Small projects

Disadvantages

- 1) No feedback
- 2) No experiments
- 3) No multiple teams
- 4) High risk (priority)
- 5) 60% effort

V-3 (SDLC) Software Development Life Cycle.



V-3 (SDLC) Software Development Life Cycle.

V-4 cont → waterfall (classical)

Advantages

- 1) Basic Model
- 2) Simple & Easy
- 3) Small projects

Disadvantages

- 1) No feedback
- 2) No experiments
- 3) No multiple teams
- 4) High risk (rigidity)
- 5) 60% effort maintenance.

V-5 iterative waterfall model

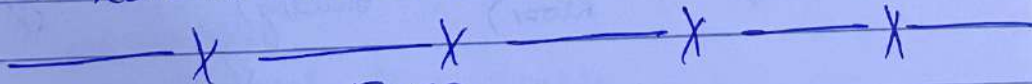
→ updated classical waterfall model with feedback in every step except feasibility study. (*feedback of developer not client)

advantages → Same as classical.

disadvantages → Same as classical except feedback.

→ Customer feedback only after delivery.

→ Less customer interaction.



PTO →

3

Feasibility Study

↓

Operation & Maintenance

[negative]

~~* diagram of V-shaped model~~

V-6 cont ...

V-6 (V-Shaped Model)

→ Also known as :-

- ① Verification Model :- Step by step verification during product making.
- ② Validation :- Validating whether the product we built is right.

→ ✓ i testing is done parrallely or after every phase in V-shaped model?

→ ① Verification Phase

→ Requirement Analysis, System design, architecture design, Module design.

eg:- Building (System) → Floor (Architecture) → room (Module)

→ ② Validation Phase

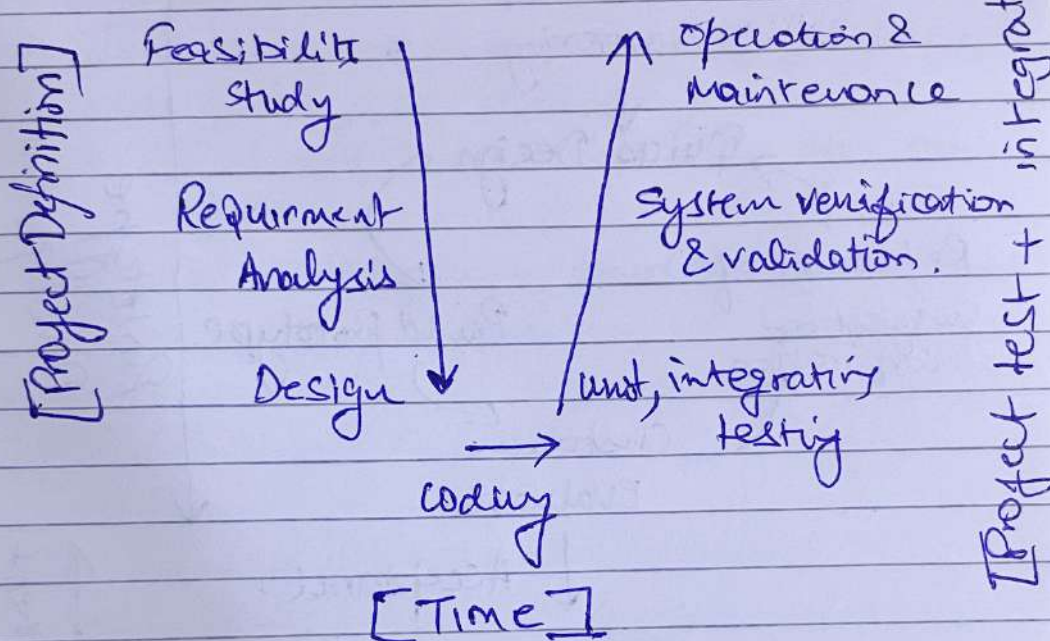
→ unit testing (Module / room), integration (room / floor), system (whole building), acceptance (validating)

Diagram in PTO →

V-6 (V-shaped Model)

V-6 cont ----

* Diagram of V-shaped Model



Advantages

- Time Saving
- Good project understanding
- Every component is testable
- Progress can be tracked.
- Proactive defect tracking.

Disadvantage

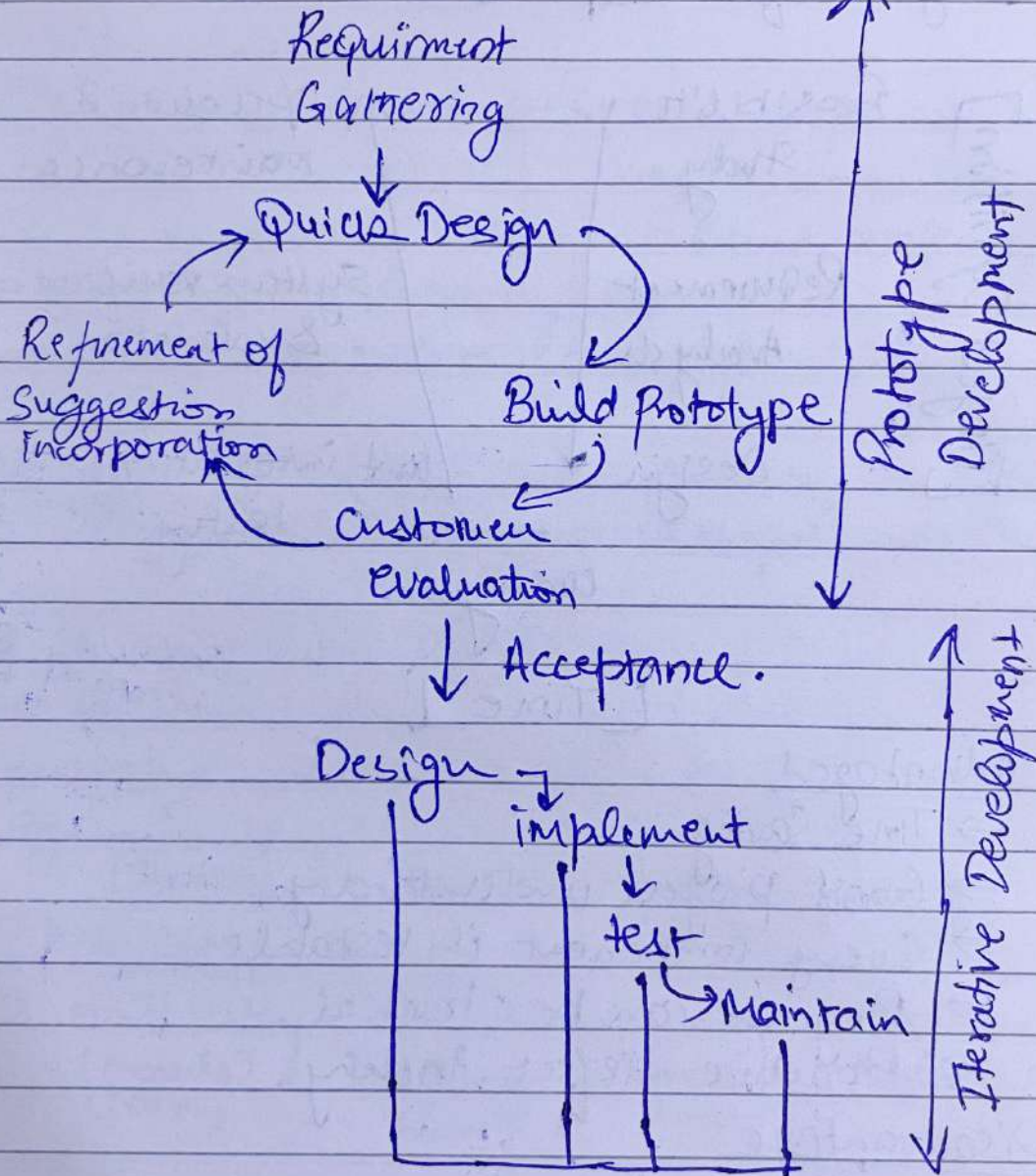
- 1) No feedback by customer.
- 2) Risk Analysis not done.
- 3) Not for big projects.

—X—X—X—PTO—X—

→ Best for customer with no clear idea.
→ It is a throwaway model.

V-7 Prototyping Model

V-7 Prototyping Model



PTO →

Requirement Gathering

V-7 Prototyping Model

V-7 cont

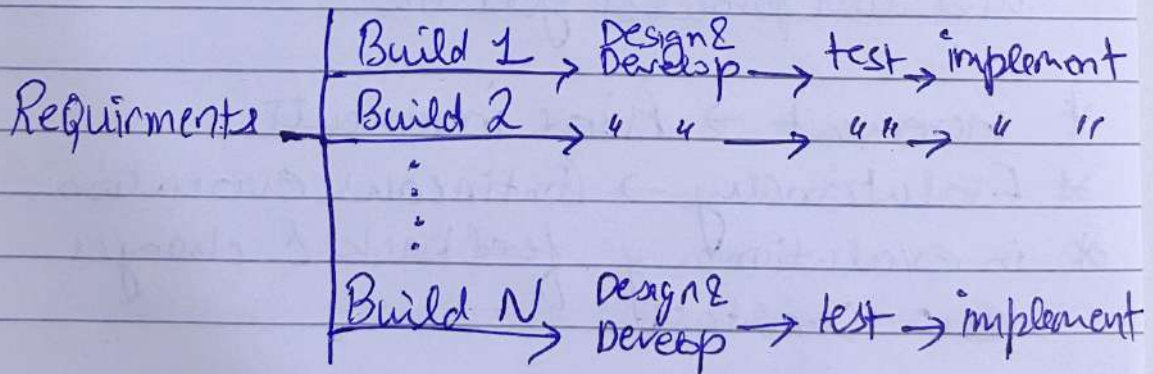
Features of prototype model

- Best for customer with no clear idea.
- It is a throwaway model (if idea not good you replace)
- If client not good technically you can suggest.
- Repeated throwaway model can increase cost of development.

— X — X — X — X —

V-8 Incremental Model

- it is a module by module Model.
- Each module is made and checked instead of making all the project in one go.



PTo →

Advantages

V-9 cont...

V-8 cont...

Features of Incremental model

- Module by Module working.
- Customer interaction maximum.
- Large Projects.
- Early release product demand.
- Flexible for changes.

— X — X — X —

V-9 Evolutionary Model

- Combination of iterative & incremental model.
- implement few features, deliver to customer and this process goes on.
- * increment → fixed increments.
- * Evolutionary → Continuous evolution.
- * in evolutionary feedback & changes are constant.

P.T.O →

V-8 cont.

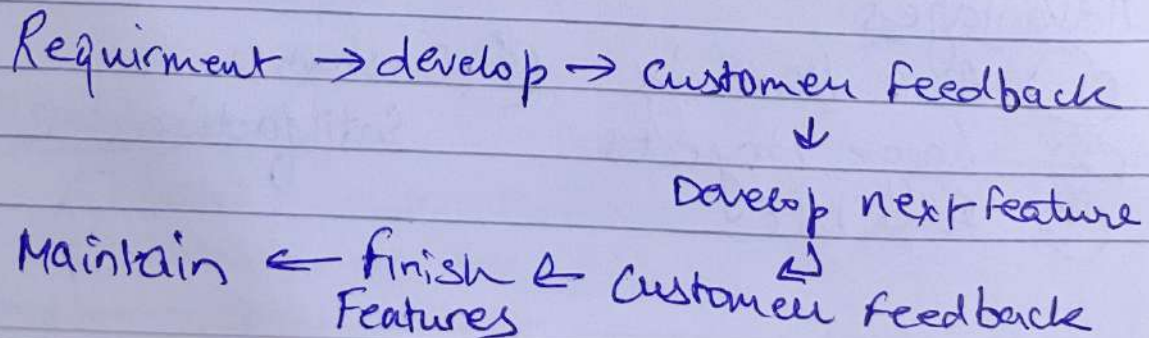
Advantages

- Customer requirement are clearly specified.
- Risk analysis is better.
- it support changing environment.
- the project starts quickly hence builds trust.
- Better suited for mission-critical projects.

Disadvantages

- ① Not for smaller projects.
- ② Cost
- ③ Highly skilled resources are required.

Flowchart

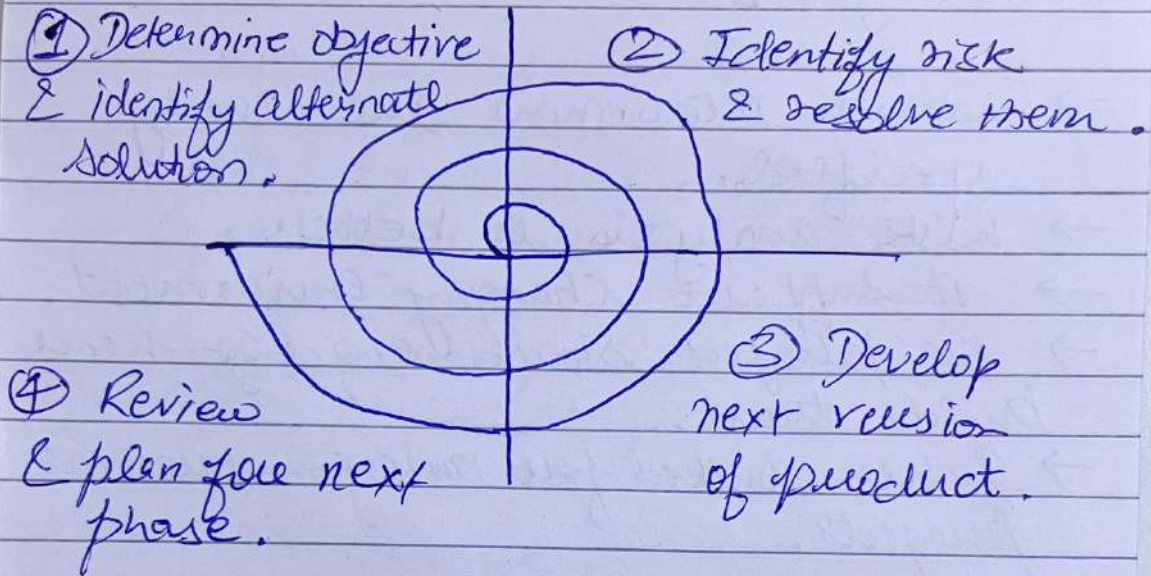


① Complex

① Disadvantages

V-10 cont

V-10 Spiral Model (Best for Risk)



- * Good Risk handling.
- * Radius of spiral = cost.
- * Angular Dimension = Progress.
- * Spiral also called Meta Model because it combines waterfall, iterative, and prototype can be created for Risk Analysis.

Advantages

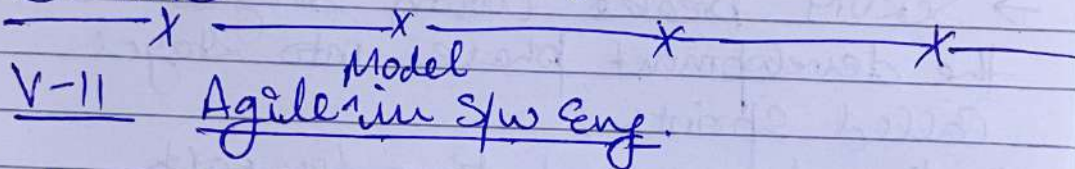
- ① Risk Handling
- ② Large Projects
- ③ Flexible
- ④ Customer Satisfaction.

V-10 Spiral Model (Best for Risk)

V-10 cont

Disadvantages

- ① Complex
- ② Expensive
- ③ Too much risk analysis
- ④ Time



V-11 Agile in S/W Eng.

Agile (Move Quickly)

Large Project
↓

Small chunks (release)
↓

Customer feedback
↓

Enhance
↓

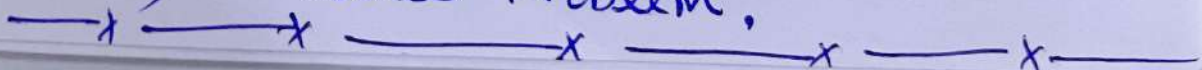
Re-release

Advantages

- ① Frequent delivery.
- ② Face to face communication with client.
- ③ frequent changes.
- ④ Least Time.

Disadvantage

- 1) Less Documentation.
- 2) Maintenance Problem.



- * Scrum Master: - Person who help team follow Scrum properly & remove obstacles.
- * Product Owner (Client): - what needs to be built and in what priority.

V-12 SCRUM Model (Everyone is equal) (no hierarchy)

- It is the most popular agile methodology.
- SCRUM is a lightweight, iterative and incremental framework.
- SCRUM breaks down stages the development phase into stages called sprints.
- the development time for each sprint is maximized & dedicated (manage one sprint at a time).
- SCRUM Team has Scrum master & product Owner with constant communication.

Keywords

- * Backlog: - Prioritized list of all work / needs to be done in the project.
- * Sprint: - Fixed time (1-2 week) to complete a set of task.
- * Daily Scrum: - daily 15 min meeting where team discuss progress and blockers.

→ It is the most popular agile method -
V-12 Scrum Model (Everyone is equal)
Primarily on

- * Scrum Master :- Person who help team follow scrum properly & remove obstacles.
- * Product Owner (client) :- what needs to be built and in what priority.

V-12 cont

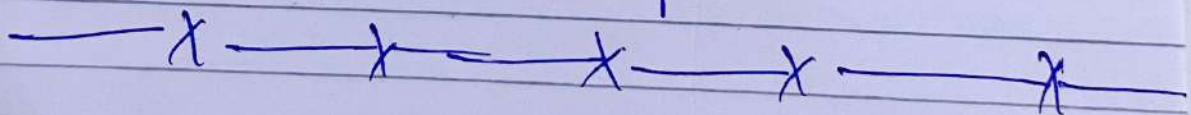
SCRUM

Advantages

- Freedom & Adaption
- High quality, low risk product.
- Reduce development time upto 40%
- Customer satisfaction.
- Review current sprint before moving to new.

Disadvantages

- Efficient for small team size.
- No changes in the sprint.



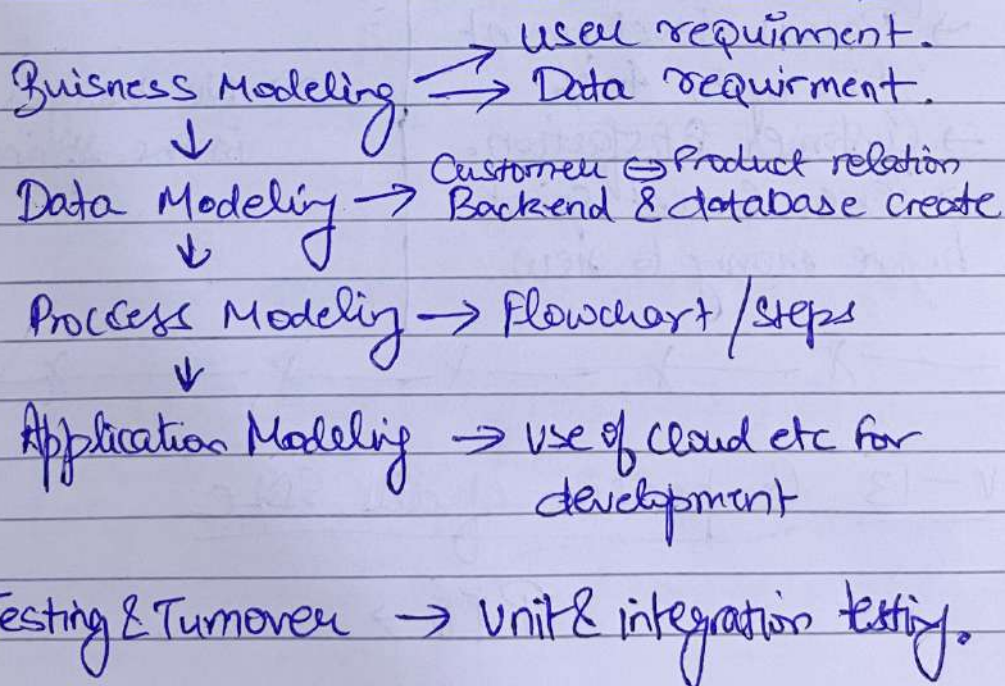
V-13 Comparision of all SDLC

V-13 Comparison of all SDLC Models

RAD Model (Rapid Application Development)

- Best for time bounded projects.
- Quick Prototyping.
- Iterative Development.
- Incremental Releases.
- User involvement
- Time boxing
- Parallel Development.

Team 1 = Team 2 = Team 3



V-13 Comparison of all SDLC Models

Comparison of SDLC Models

- ① Classic Waterfall Model
 - Basic → Rigid → Inflexible
 - Not for real project.
- ② Iterative Waterfall Model
 - Basic, Feedback given. (User clear requirements)
- ③ Prototype Model
 - User not clear about requirement, costly, no lock on requirement, high user involvement, reusability.
- ④ Incremental Model
 - Module by Module delivery
 - After each module feedback given.
 - Early to test & debug.
- ⑤ Evolutionary Model
 - Same as Incremental but requirement is not locked.
 - Constant updates.
- ⑥ RAD Model
 - Time & cost constraints.
 - User involvement at all levels.
 - Reusability.

PTD →

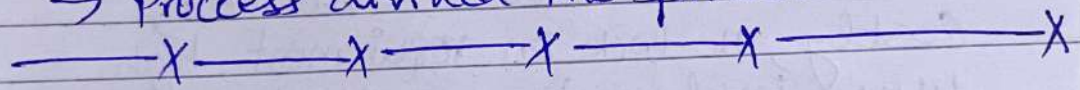
- ① Observation Report
- ② Questionnaire
- ③ Use cases
- ④ User stories
- ⑤ Prototype
- ⑥ Mind mapping
- ⑦ Role play
- ⑧ Prototype

⑦ Spiral Model

- Risk (best management)
- Not for small projects.
- No early lock on requirements
- Less experience can work.

⑧ Agile Model

- Flexible → advanced → parallel
- Process divided into sprints.



8-4-26

Wednesday V-14 Software Requirements

- Features & Functionality of target System.
- Tell what system should do.
- Requirement engineering (RE)
- Define → Documenting → maintain requirement in Eng. design.

4 Steps

- ① Feasibility study (possible → cost future etc)
- ② Requirement gathering
- ③ (SRS) Software Requirement Specification.
- ④ Software requirement Validation.

- ⑦ Spiral Model
→ Risk (best management)
→ Not for small projects.
→ No early look on requirements
→ Less experience can work.
⑧ Agile Model
→ Flexible & advanced → parallel
- ⑨ V-shaped Model

- Tools for RE
- | | |
|----------------------|------------------------|
| ① Observation Report | ⑤ Requirement workshop |
| ② Questionnaire | ⑥ Mind Mapping |
| ③ Use cases | ⑦ Role play |
| ④ User stories | |
| ⑧ Prototype. | |

V-15 Functional Vs Non-functional Requirements in SRS

Functional :- Working aspect of software.

Non-functional :- Characteristics of target software. (~~Security, Storage, Cost, maintenance~~).

- ① Security, ② Storage, ③ configuration, ④ Performance, ⑤ Cost, ⑥ interoperability, ⑦ Flexibility, ⑧ Disaster recovery, ⑨ Accessibility.

V-16 Software Requirement Specification (SRS)

→ SRS [Software to be developed]

PTO →

- ③ System Features & Requirements
- 3.1 Functional requirement
 - 3.2 Use cases (user-system interaction)
 - 3.3 External interface requirements
 - 2.3 Constraints, assumptions & dependencies - (Rely on others)
 - 2.4 User characteristic.

V-16 cont - -

- SRS lays out both functional & non functional requirements of software to be developed.
- SRS include set of use cases that describe user interaction that the software must provide to the user perfect interaction.

SRS Structure

① Introduction

- 1.1 Purpose
- 1.2 intended Audience
- 1.3 Scope
- 1.4 Definitions
- 1.5 References

② Overall Description

- 2.1 User interfaces
- 2.2 System interfaces

- SRS include set of use cases that describe user interaction that the software to be developed.
- SRS lays out both functional & non functional requirements of

V-16 cont - -

- 2.3 ^(limit) ^(guess) Constraints, assumptions & dependencies. (rely on others)
- 2.4 User characteristic.

③ System Features & Requirements

- 3.1 Functional requirement
- 3.2 Use cases (user-system interaction)
- 3.3 External interface requirement
- 3.4 Logical database requirement.
- 3.5 Non-functional requirements.

④ Deliver for Approval.

—X—X—X—X—X—

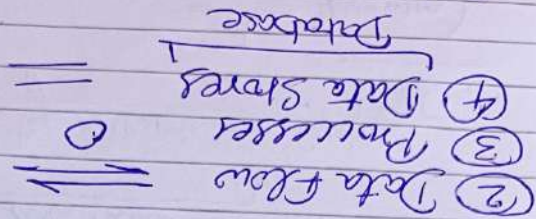
V-17 User Requirements & Egs (end-user, not client)

- Simple interface.
- Quick response (No waiting)
- Effectively handling error
- Customer support.

PTO →

Source - Entity that supply data to the system.

External Entities



V-18 cont. ---

V-17 User requirement Specification
on user requirement document (URD)

→ Predefined documents (URS)
& (URD) predefined specify
what software be able to do.

→ Contractual agreement.

V-8 Data Flow Diagrams (DFD)
(bubble chart)

→ A graphical tool, useful for communi-
cating with users, managers and
other personnel.

→ Analyse existing & proposed systems.

→ Basically show how the data
moves in a system.

DFD elements

① Source / Sinks (external entities)
~ user, etc.

V-17 User requirement specifications (URS)
 On user requirement document (URS)
 → predefined documents (URS) & (URS) ~~predefined~~ specify what software is able to do.

V-18 cont ----

- ② Data Flow $\rightleftarrows$
- ③ Processes $\circ$
- ④ Data Stores $=$
 Database

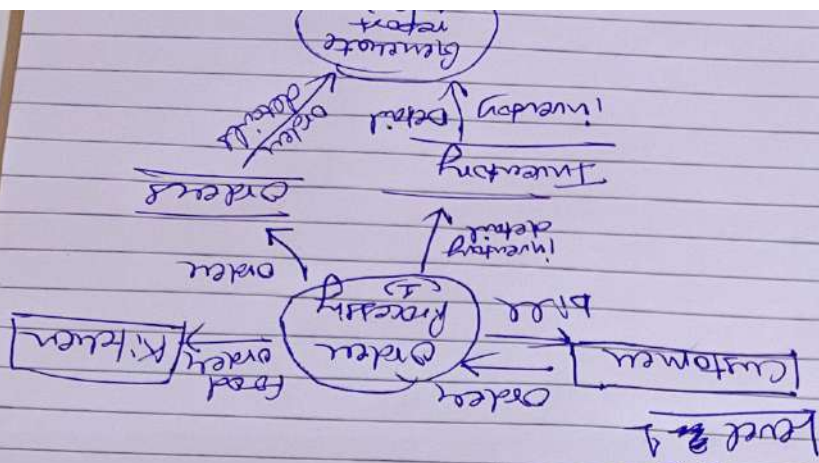
External Entities

Source :- Entity that supply data to the system.

Sink :- Entity that receive data from the system.

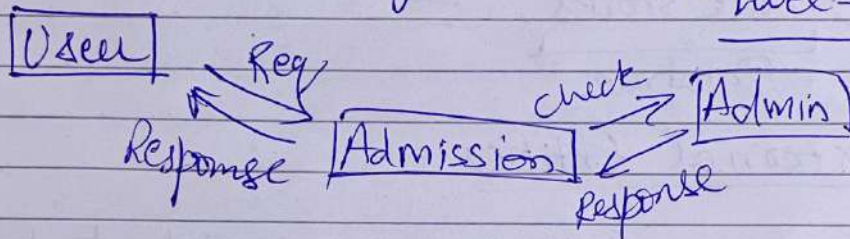
Rules of data flow

<u>Data can flow from</u>	<u>Data cannot flow from</u>
→ External entity to process. $(E \rightarrow P)$	→ External entity to external entity.
→ process to external entity. $(P \rightarrow E)$	→ External entity to store
→ Process to store & back. $(P \leftrightarrow DB)$	→ Store to external entity.
→ process to process. $(P \rightarrow P)$	→ Store to store



V-19 Level of Data Flow Diagram (DFD)

DFD for University Admission System Level-0



Level-0 (Food order System)



Level-1 (More detailed)

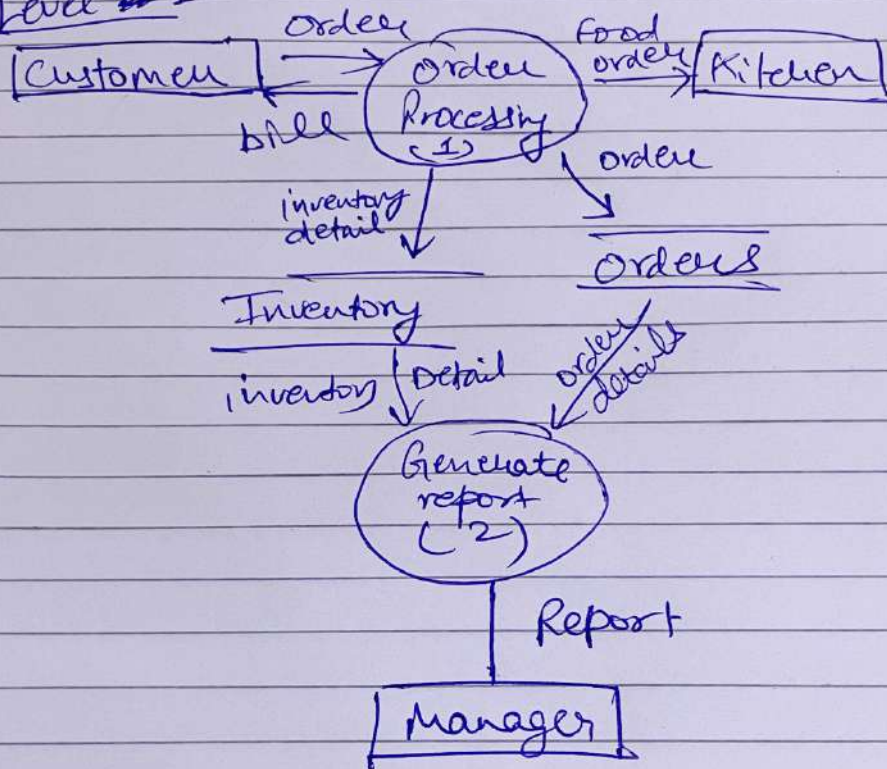
Customer

PTO →

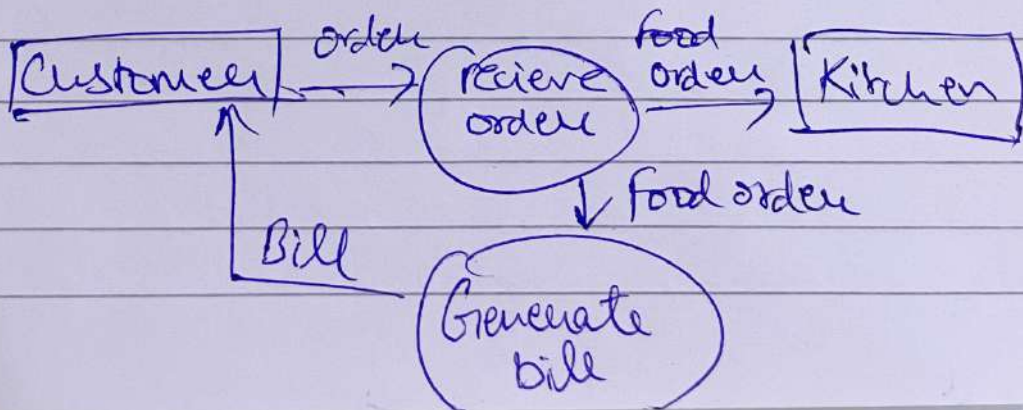
V-19 Level of Data Flow Diagram (DFD)

V-19 cont...

Level 1



Level -2



→ Main goal is to enable a group of developers to work towards a common objective

→ SPM is an art of planning & leading software projects.

V-20 Logical Vs Physical DFD

Logical DFD → Describe what data is stored and relationship b/w the data.

Physical DFD → Describe how data is actually stored in memory / storage.

V-21 Functional Vs object oriented

Functional oriented

→ System is designed from functional viewpoint.

→ Top-down decomposition.

→ Divide & conquer approach.

→ DFD is used.

Object oriented

→ System is viewed as a collection of objects.

→ bottom up approach

→ Bottom up approach

→ UML is used.

V-20 Logical vs Physical DFD

Logical DFD → Describe what data is stored and relationship b/w the data.

Physical DFD → Describe how data is actually stored in memory / storage.

V-22 SPM (Software Project Management)

- SPM is an art of planning & leading software projects.
- Main goal is to enable a group of developer to work toward successful development of project.
- Project manager (administrator)
- SPM complexities

① Changeability ② Complexity ③ Uniqueness
④ Multiple solution.

SPM (responsibilities)

- ① Planning
- ② organizing
- ③ Staffing
- ④ Directing
- ⑤ Monitoring
- ⑥ Controlling
- ⑦ innovating
- ⑧ representing.

SPM (Skills)

- ① Managerial skill
- ② Technical skill
- ③ Problem Solving
- ④ Copying
- ⑤ Conceptual
- ⑥ Leadership
- ⑦ Communication.

Risk Management plan :- It is a docu-

Risk :- uncertain event if occurs has positive or negative effect on projects objective.

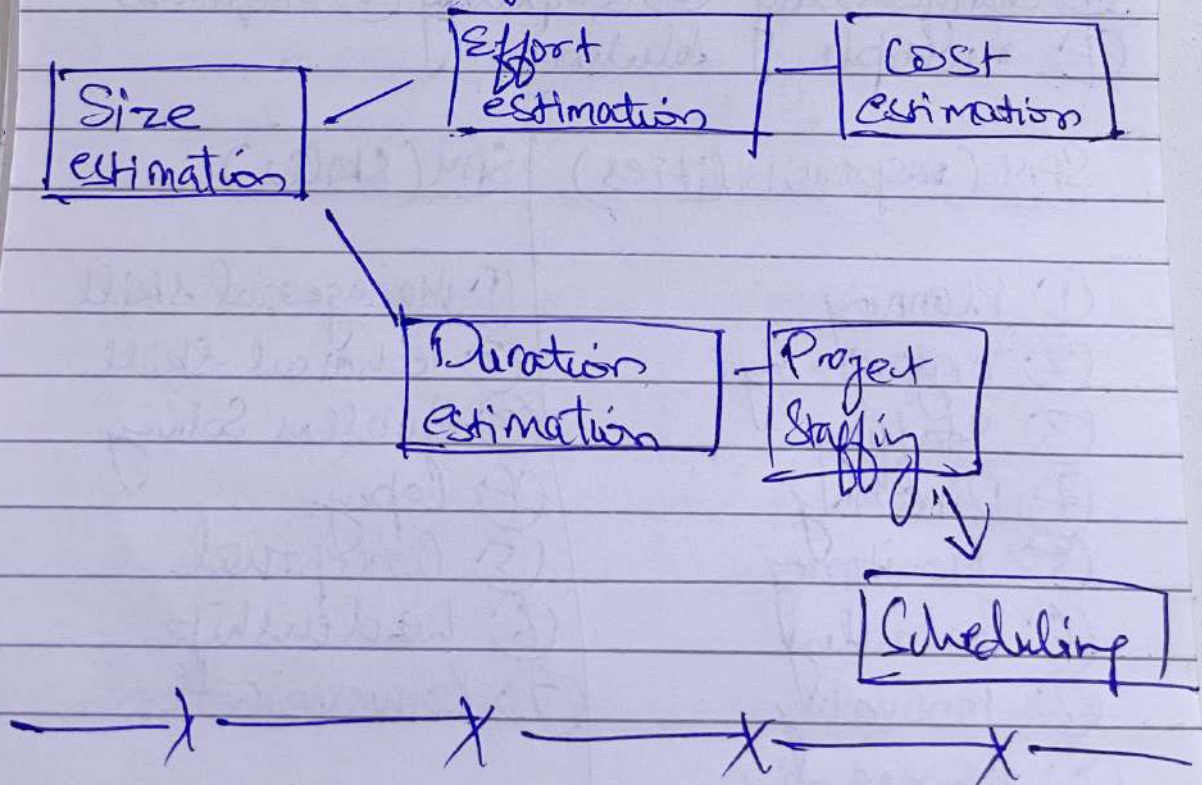
Risk Identification

V-22 cont →

Project Planning

- ① Estimation (Cost, Duration, effort)
- ② Staffing (staff organization, staff plan)
- ③ Scheduling manpower
- ④ Risk Management.
- ⑤ Miscellaneous (Quality assurance, configuration & installation plan).

Precedence Ordering



Project Planning

V-22 Cont →

V-23 Risk identification

Risk :- uncertain event if occurs has positive or negative effect on project's objective.

Risk Management plan :- It is a document that a project manager prepare to foresee risk, estimate impact & define responses to risk.

Reactive Risk Management (Damage control)
→ Manage risk after they occur.

Proactive " " (Prevention)
→ identifying risk before they occur.

Types of Risks

Business Risk :- Building project no one needs.

Technical Risk :- Concerned with quality, design, interface, maintainance problem.

Project Risk :- Concerned with schedule, cost, resources, customer related issues.

— X — X — X — X —

V-24 Risk Assessment

- Rank risk in terms of damage.
- Probability of Occurrence of each risk.
- Determine the severity value.

$$RA = P \times S$$

X X X X

V-25 Risk Control vs Risk Mitigation

- Risk Control reduce probability of risk events.
- Risk mitigation reduce impact of risk events.

Strategies

- ① Risk Avoidance
- ② Risk Transfer
- ③ Risk reduction
- ④ Risk Monitoring.

X X X X

V-24 Risk Assessment

V-26 Basic COCOMO & intermediate COCOMO

Project estimation $\rightarrow$ Size $\rightarrow$ Duration
 $\rightarrow$ Effort $\rightarrow$ cost

Estimation techniques

- ① Empirical estimation techniques.
- ② Heuristic estimation techniques.
- ③ Analytical estimation techniques.

① cost, time, guess (empirical)

Ⓐ expert judgement (alone)

Ⓑ Delphi cost estimation (team)

② Heuristic

$$\text{estimated} = C1^{2*} \text{Parameter}$$

COCOMO (constructive cost Model) (LOC) ^{line of code}

① <u>Organic</u> 2-5 KLOC Small project No tight deadline	② <u>Semi-detached</u> 50 - 300 KLOC Medium Project Medium deadline	③ <u>Embedded</u> over 300 KLOC Significant tight deadline
--	--	---

~~(person and computer) unit~~ →
V-26 cont...

Person Month. (Effort put by a person in 1 month).

but LOPM doesn't mean same.
It's unit of effort.

Basically, COCOMO Model estimate software projects, effort, time & cost based on LOC (line of code). or KS LOC (Kilobits line of code).

Merits

Eg of COCOMO Model
(MIS) Management information system.

① quick, rough estimate of software cost.

Demerits

① Does not account for experience, quality or hardware liabilities.

② Due to above reason accuracy is limited.

— x — x — x — x —

V-26 Cont...

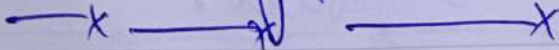
V-27 CPM (Critical path Method)



(V-28) Verification Vs Validation

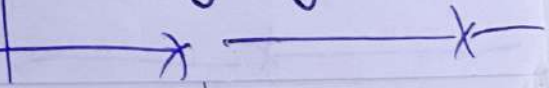
Verification

- "Are you building right"
- Future part matches Previous part.
- Done by developer
- Concerned where the error is.
- Involve Review, inspection, unit testing & integration testing.
- Static & Dynamic testing.

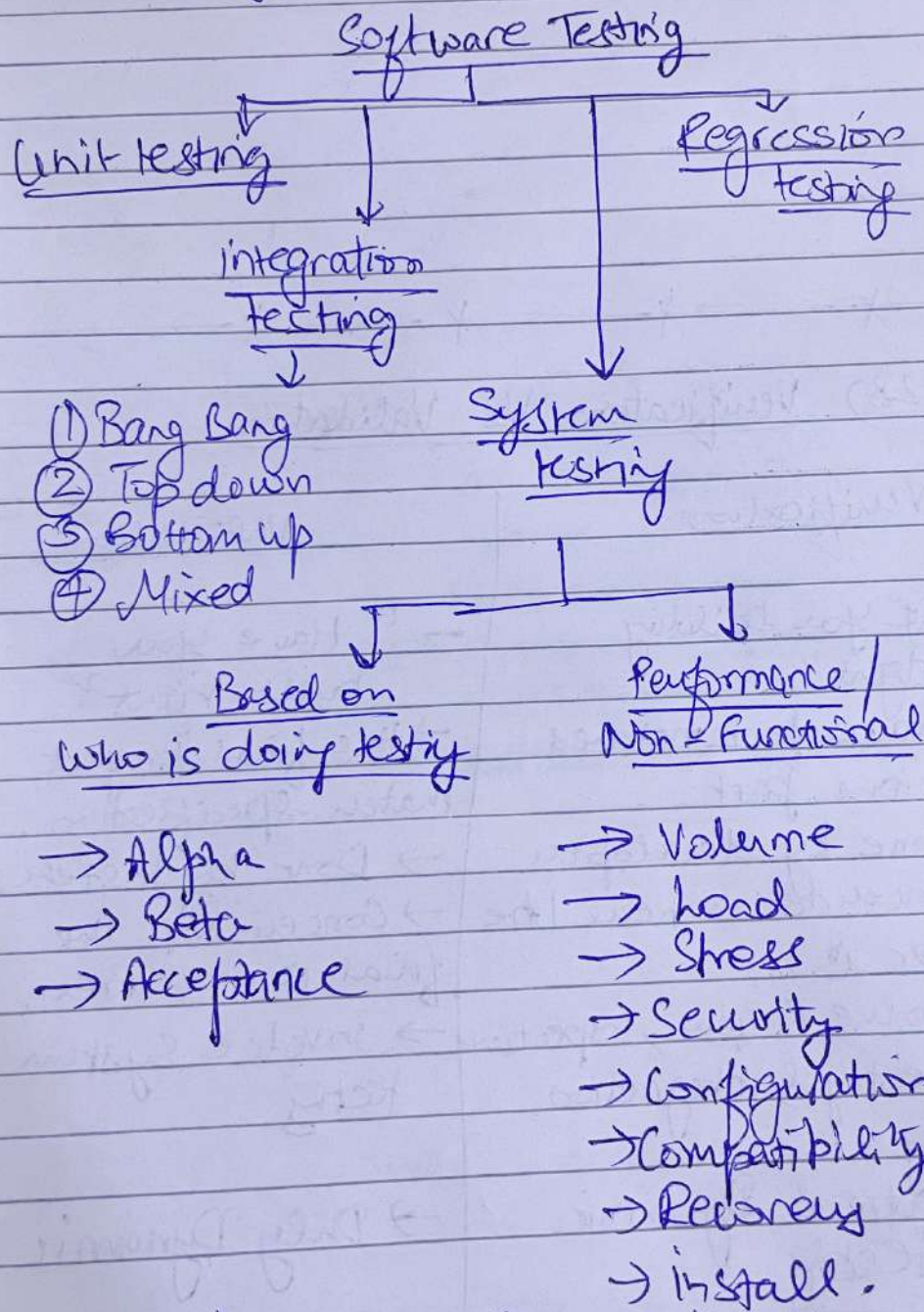


Validation

- "Have you built right"
- The final product match specification.
- Done by tester.
- Concerned the final is error free.
- Involve System testing.
- Only Dynamic



V-29 Types of testing



— x — x — x — x —

Software Testing

V-29 Types of testing

V-30 Error Seeding in Testing

S = Total errors seeded in code

s = total seeded error detected

N = ^{during testing} Total Errors

n = Total non seeded error

$$\frac{n}{N} = \frac{s}{S} \quad \boxed{N = \frac{n \times S}{s}}$$

$$UD(\text{undetected error}) = \frac{n \times (S - s)}{s}$$

— X — X — V — X —

V-31 2 imp MCQ on S/W Eng.

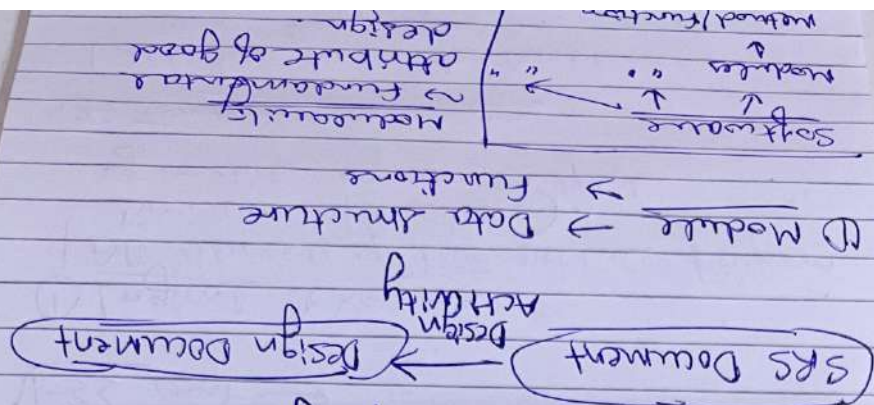
① Best Risk Model $\rightarrow$ Spiral

② $\frac{LOC}{FP} = 50$

$$LOC = 50 \times 500 \\ = 25000$$

$$E = \frac{2.5}{4} (25)^{1.0} \rightarrow 62.5 \text{ pm}$$

— X — X — X —



V-32

Cyclomatic Complexity

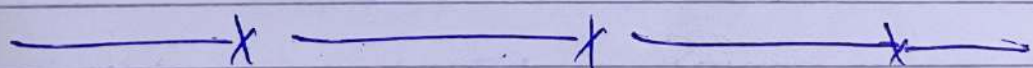
White box

- ① Path coverage technique
(test in each code)

Black box

- ① Equivalence class testing
(divide into groups)
- ② Cause effect graph
(cause-effect relationship)
- ③ State based testing.
(Start → 1st → 2nd)
↓
Success

→ Cyclomatic complexity of flow graph is $[R+1]$.



Cyclomatic Complexity

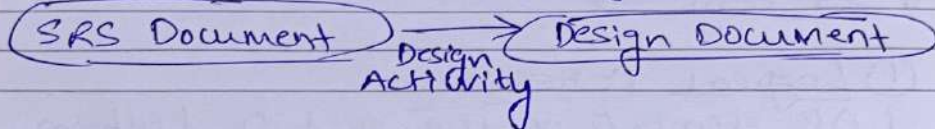
V-32

White box

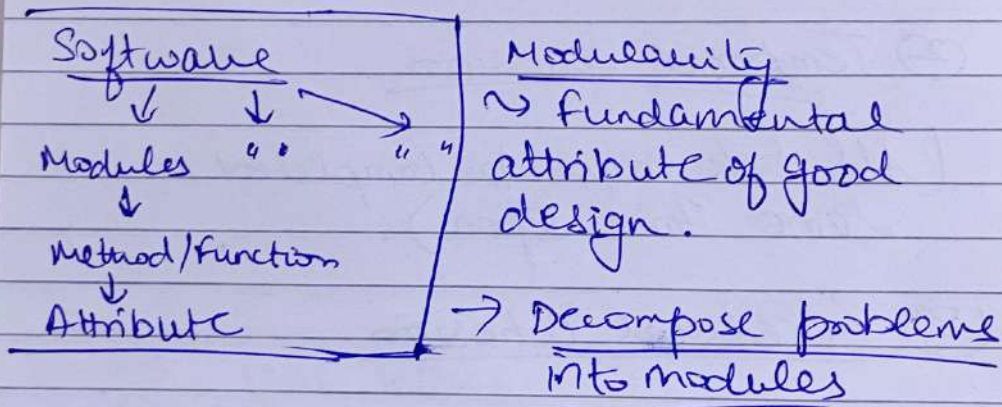
(1) Path coverage
Technique

Black box
(2) Equivalence
Class testing
(divide into

V-33 Cohesion & Coupling



(1) Module → Data structure
→ Functions



- (1) Modules are almost independent of each other.
- (2) divide & conquer principle.

Cohesion (togetherness) [measure of strength]
→ within 1 module.

Coupling (interaction b/w modules)
[Measure of dependencies]

V-33 cont →

① Logical Cohesion

[All elements of the module perform similar operation].

eg:- data input, output.

② Temporal Cohesion

[All task must be completed in same time span].

③ Procedural Cohesion

[All elements of module group together, which are executed sequentially].

④ Communicational Cohesion

→ When elements are grouped together, elements are executed sequentially and work on same data (information).

P.T.O →

① Logical cohesion
(All elements of the module perform
a similar operation).
g:- data input, output

② Temporal cohesion

V-33 cont →

V-33 cont

Sequential cohesion :- All elements
of a module form different sequence
of output from one element is
input to next.

Fan-in :- How many module invoke
a given module!

Fan-out :- number of module controlled
by given module.

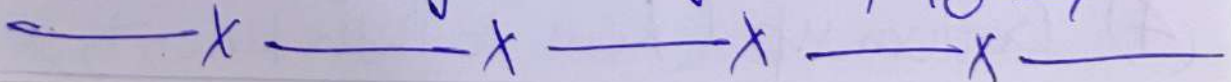


V-34 Unit testing

→ Unit testing is the process of testing
individual units of functionality.

→ Unit testing is the process of checking
small pieces of code to ensure
that the individual part of
the program works properly
on their own.

→ Unit testing done by developer
PTO →



V-35 Integration Testing (combination of unit test)

→ This testing is done to ensure that the system works with compliance of specified functional requirement.

→ It occurs after unit testing & before system testing.

→ Types of integration testing

(1) Big - Bang

(2) ~~&~~ Mixed (Sandwich)

(3) Top-down

(4) Bottom up

(1) Testing after combining unit test is big-bang.

(2) Mixed is combination of top-down & bottom up.

(3) Top-Down (high to low priority testing)

(4) Bottom up (low to high priority)

V-35 Integration Testing (combination of unit test)

→ This testing is done to ensure that the system works with compliance of specified functional requirement.

→ It occurs after unit testing &

V-36 System testing

→ level of testing that validate the complete and fully integrated software product.

→ It checks end-to-end software matching the specification.

→ It is a black box testing.

→ System test divides based on who is doing testing.

→ Also based on : Functional & Non-Functional.

— X — X — X — X —

V-37 Types of System testing

① Performance testing :- Test to carry speed, scalability, stability and reliability of software product.

② Load Testing :- Software product testing under ~~heavy~~ load.
Expected/Normal PTO →

White box design technique

① Statement coverage :- Test all the lines.

② Branch coverage :- Test each if/else path.

③ Stress testing :- Testing performed under extreme load.

④ Scalability testing :- Testing performed on the software checks if system can increase or decrease its capability (users) without performance issues.
eg:- system works after adding servers.

V-38 White box testing

→ Method of software testing that test internal structure of an application.

→ Mainly done at Unit testing
Sometimes at integration testing.

→ Some tools are :-

→ Veracode, Nunit, Cpp Unit.

PTO →

V-37 cont →

③ Stress testing :- Testing performed under extreme load

④ Scalability testing :- Testing performed on the software checks if system can increase or decrease its capability (users)

V-38 cont - - - →

White box design technique

- ① Statement Coverage :- Test all the lines.
 - ② Branch Coverage :- Test each if/else outcome.
 - ③ Path Coverage :- Test all possible path in the program.
 - ④ Condition Coverage :- Test condition inside decision
- Statement.
 - ⑤ Loop Testing :- Test loop for different iterations.
- X — X — X — X —

V-39 White box Vs Black box

<u>White</u>	<u>Black</u>
→ Performed by developer.	→ Performed by testers
→ Aware how S/w work. (internally)	→ Not aware how S/w work (internally).
→ Programming Knowledge needed.	→ No need to know programming.

V-39 cont -->

White box

→ Should know internal design of code.

→ Techniques:-

(A) Control-flow testing.

(B) Data-flow testing

(C) Branch testing

(D) Decision testing.

(E) Path testing.

~~At~~

→ Mainly performed at Unit testing.

Black box

→ No need to know internal design.

→ Techniques:-

(A) Decision table testing.

(B) All-pair testing

(C) Equivalence partitioning.

(D) Boundary Value Analysis

(E) Cause-effect graph

→ Performed at System testing.

— X — X — X —

→ Calculate total number of executed statement in source code out of source code.
 → Covers :- Dead code, unused code, branches.
 V-41 Condition coverage in white box testing.

V-39 cont -->

White box

- Should know internal design of code.
- Techniques :-
 - (A) Control-flow testing.
 - (B) Data-flow testing
 - (C) Branch testing
 - (D) Decision testing.
 - (E) Path testing.

→

→ Mainly performed at Unit testing.

Black box

- No need to know internal design.
- Techniques :-
 - (A) Decision table testing.
 - (B) All-pair testing
 - (C) Equivalence partitioning.
 - (D) Boundary Value Analysis
 - (E) Cause-effect graph
- Performed at System testing.

V-40 Statement Coverage technique

- design white box test cases.
- This technique involves coverage of all statement of source code at least once.

V-40 Cont →

→ Calculate total number of executed statement in source code out of total statement present in source code.

→ Covers :- Dead code, unused code & branches.

—X—X—X—X—X—

V-41 Condition coverage in white box testing

→ It is used to cover all conditions.

→ Condition coverage is known as predicate coverage

→ Covers can True/False.

—X—X—X—X—X—

V-42 Data Flow testing in white box

→ Focuses on where variables are defined. (how many variable in total)

→ Focuses on where variables are used. (how many times variable used)

PTO →

→ Used to modify and correct errors after delivery of the software product -

Software Maintenance

Engineering

Corrective maintenance in situ

V-42 Advantage of Data Flow Testing

- A variable that is declared but never used within the program.
- A variable used but never declared.
- A variable declared multiple times before used.
- Deallocating a variable before it is used.

— X — X — X — X —

V-43 Boundary Value Testing

- Black box testing technique.
- Helpful to detect errors or threats that happen at boundary value rather than center.

(40) Boundary Value Test Case (80)

Invalid test case (min value - 1)	Valid	Invalid
(39)	(40), (41), (79), (80)	(81)
X	✓	X

- A variable that is declared but never used within the program.
- A variable used but never declared.
- A variable declared multiple times before used.

V-42 Advantage of Data Flow Testing

V-44 ~~Perfective~~ Preventive, Adaptive, Corrective maintenance in S/W Engineering

Software Maintenance

- Used to modify and correct errors after delivery of the software product.
- Maintenance role plays the highest role in SDLC.
- Error Correction
- Deletion of delete outdated technology.
- Enhancing capabilities.
- Optimization

4 type S/W Maintenance

① Corrective Maintenance :- Fixing bugs after software is delivered.

Eg:- App crash → developer fix.

MTBF is avg. time for a system to perform its normal operation b/w failure.

Mean time b/w failure (MTBF)

V-44 cont

② Adaptive Maintenance

→ Updating software to work in new environment.

Eg: - updating app for new android version.

③ Perfective Maintenance

→ improving performance, UI or features.

Eg: - Adding dark mode or improving speed.

④ Preventive Maintenance

→ Make changes to avoid future problems.

Eg: - Reducing bug by code refactoring.

—————X—————X—————X—————

PTO →

② Additive Maintenance

V-44 cont

V-45 MTBF vs MTTR

Mean time b/w failure (MTBF)

MTBF is avg. time for a system to perform its normal operations b/w failure.

$$MTBF = \frac{\text{Sum of operational time}}{\text{Total number of failure.}}$$

Mean time to Repair (MTTR)

→ Average time required to repair a failed component.

$$MTTR = \frac{\text{Sum of downtime period}}{\text{Total number of failure.}}$$

$$\text{Availability} \rightarrow \frac{MTBF}{(MTBF + MTTR)}$$

(Ques) A machine run 24hr. failed 2 time each time take 4 hr to repair. Find MTBF & MTTR.

$$(Ans) MTBF = \frac{24 \times 11}{2} \quad | \quad MTTR = \frac{2}{2} = 1$$

→ Software tools used to design and implement applications.
 → CASE tools develop high quality, defect free

Engineering
 V-45 cont →

$$A = \frac{11}{11+1} = 11/12$$

— x — x — x — x — x —
V-46 Reverse Engineering / Ballward Engineering

- Understanding underlying source code for the maintenance & improvement of the software.
- In some cases the goal of reverse engineering process helps in Redocumentation of legacy system.

Code → Module specification → Design
 ↓

* Misused (Copyright)	* Used to check virus any from 2	Requirement Specification
— x —	— x —	— x —

PTO →

$$A = \frac{11}{12} = 0.9167$$

$$V-45 \text{ Cost} \rightarrow$$

V-47 (CASE) Computer-aided Software Engineering.

- Software-tools used to design and implement applications.
- CASE tools develop high quality, defect free & maintainable software.
- Helps in S/w development & maintenance.
- improve Quality at lower cost & increase productivity.

V-48 Performance Testing

→ Non-functional testing determine how a system perform in terms of responsiveness and stability under a proper workload.

- A performance test measure stability, speed, scalability and throughput of your product.

Pro →

→ Type of testing that verify that changes made to the system does not impact previously

Regression testing

Reaction to sudden increase & decrease in the load.

V-49

V-48 cont →

Load Testing :- Product's ability to perform under increased load.

Eg :- Maximum Capacity of a web server.

Stress Testing :- Capacity of a software to perform well at upper limits are pushed.

Eg :- Website on major sale event.

Volume testing :- Database testing to ensure it handle large amount of data.

Endurance testing :- System performance over long period of time, usually under normal load, on performance degradation over time.

Response time testing :- Evaluate time it take to respond to a request, with goal of acceptance response time for end-user.

PTO →

Load Testing :- Product's ability to perform under increased load.
Eg :- Maximum capacity of a web server.

V-48 test →

Spike testing :- Tests the product reaction to sudden increase & decrease in the load.

V-49 Regression testing

→ Type of testing that verify that changes made to the system does not impact previously working functionality.

→ Types of regression testing.

① Full Regression Testing :- Testing the entire application from start to finish after changes are made.

② Partial Regression Testing :- Testing only those part that were affected by the change.

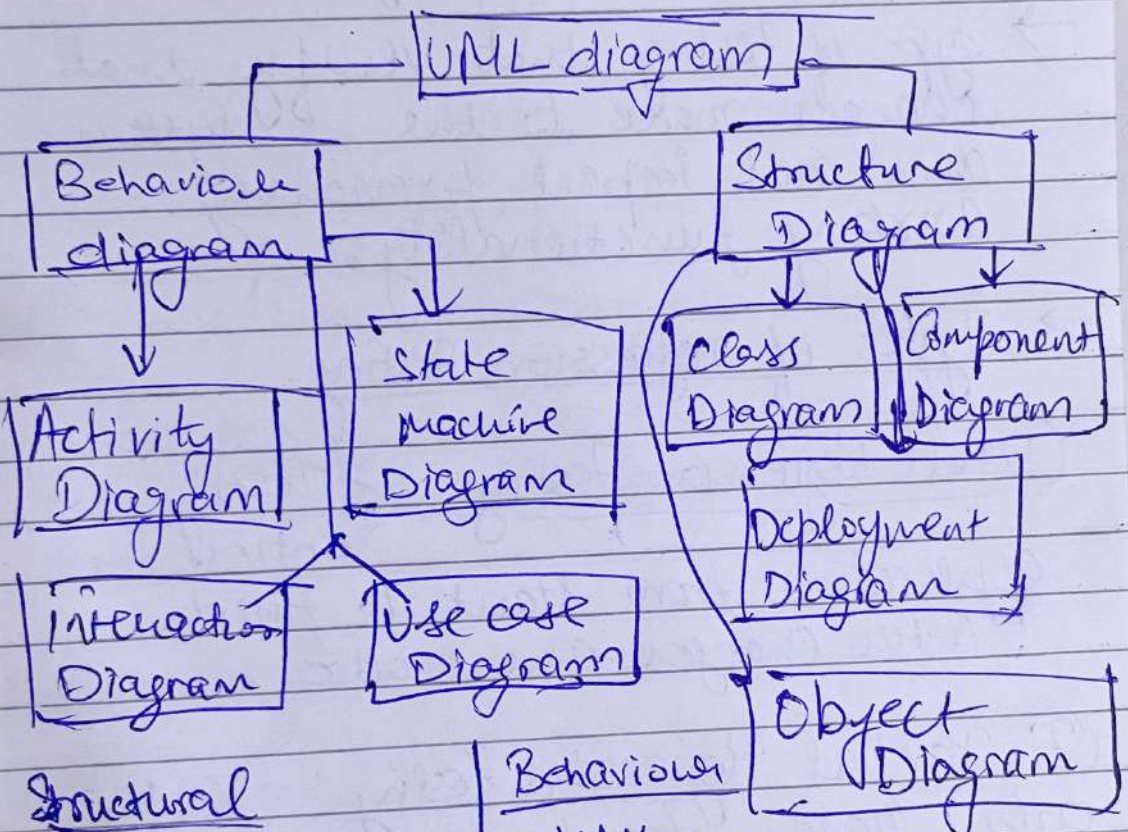
→

PTO →

→ Use case diagram model the behavior of a system.
 → Illustrate functional requirement of the system and interaction with external agent

→ UML Use case diagram in UML

N-50 (Intro to UML (Unified Modelling language))

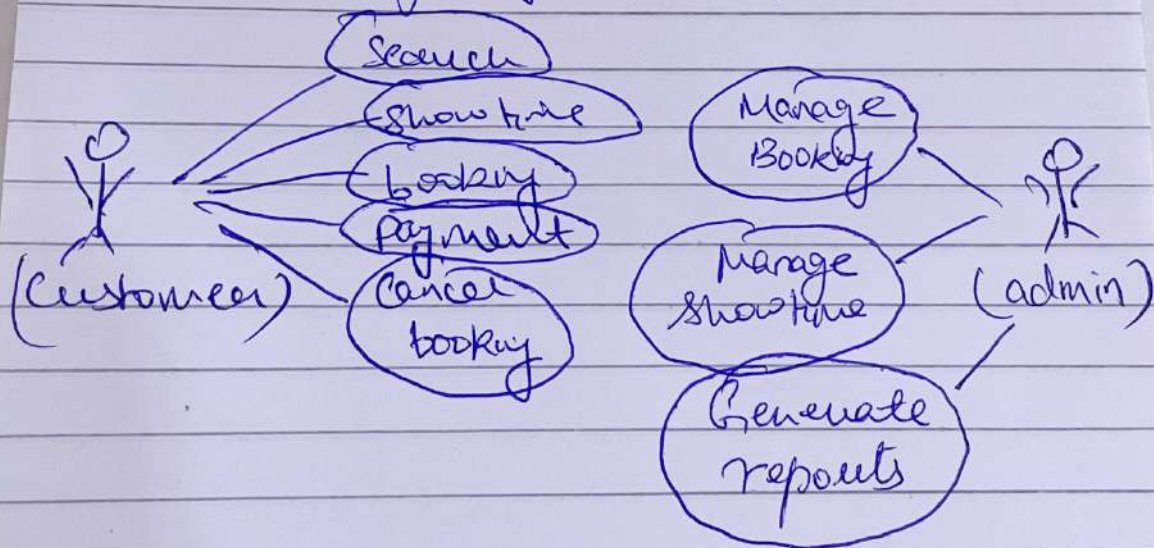


<u>Structural UML</u>	<u>Behaviour UML</u>
→ Focus on how System is built, describe system architecture, entities & properties.	→ Focus on System works.
	→ Describe how the system responds in different scenarios.
X	X

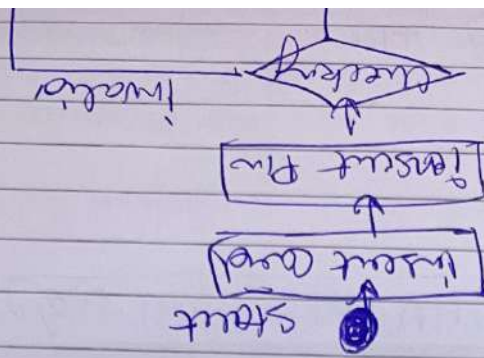
V-50 (Intro to UML (Unified Modeling Language))

V-51 Use case diagram in UML

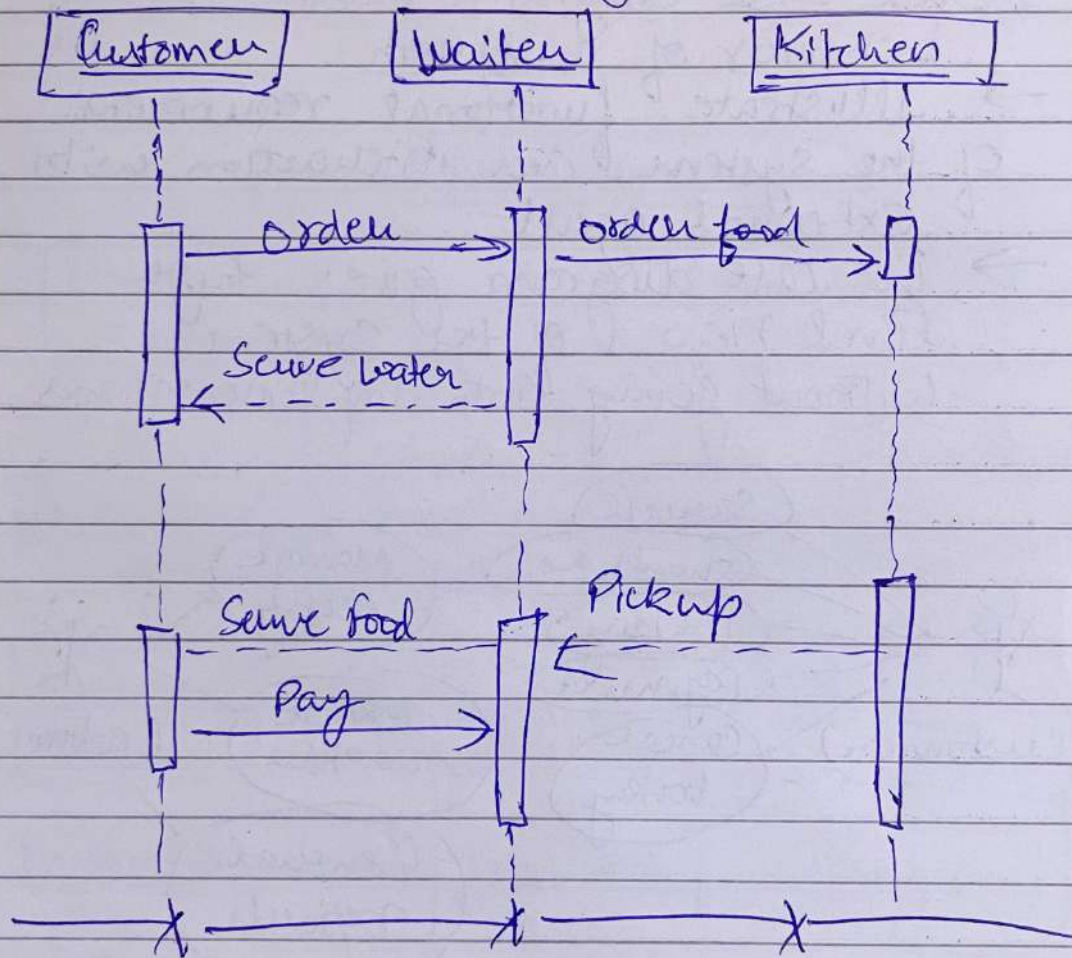
- use case diagram model the behavior of a system.
- illustrate functional requirement of the system and interaction with external agent.
- Use case diagram gives high level view of the system without going into implementations.



— X — X — X — X —



V-52 (Sequence Diagram)

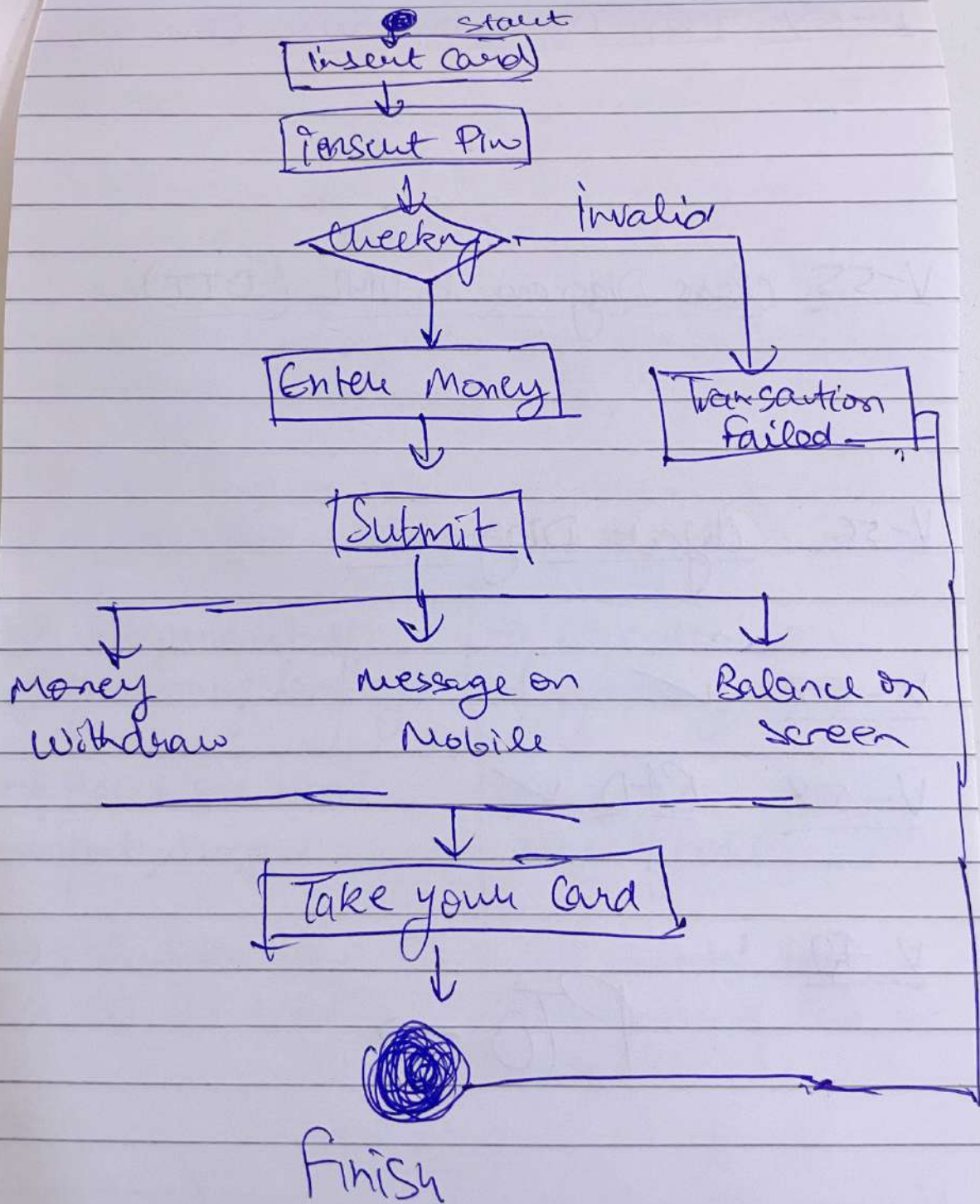


V-53 Activity Diagram

PTO →

V-52 (Sequence Diagram)

V-53 cont →



Function Point (FP) → Measure the -
 functionality delivered to user based on user requirements
 → Measure the physical size of code written in

V-59 line of code (LOC)
 Function point (FP) vs

V-54 Class Diagram in UML (Banking)

V-55 Class Diagram in UML (OTT)

V-56 Object Diagram

V-57 ✓

V-58 RAD ✓

V-59

PTO →

V-54 Class Diagram in UML (Banking)

V-59 ~~mm~~ Function point (FP) vs
Line of Code (LOC)

<u>Function Point (FP)</u>	<u>Line of Code (LOC)</u>
→ Measure the - functionality delivered to user Based on user requirements & design.	→ Measure the physical line of code written in programming language.
→ User functionality & System behavior.	→ Amount of code written.
→ Independent of programming language.	→ Dependent on programming language.
→ Based on users input, output.	→ Based on counting line of code.
→ Calculated using: EI, EO, EQ, IWF, EIF.	→ Calculated by counting no. of line in codebase.
← X — X	X — X

→ Data from system to user
eg: - Confirmation email.

→ request + response, no internal
data change.
eg: - ordered item.

(3) External inquiries (EQ)

V-60 Function Point Analysis (FPA)

→ Method to estimate size, complexity & functionality of software projects.

Unadjusted Function points (UFP) :

Calculate by multiplying each component by its weight & summing them up.

Value Adjustment Factor (VAF) :

$$FP = UFP \times (0.65 + 0.01 \times \text{Sum of GSCs})$$

GSCs → General System Characteristics

list of 5 unadjusted Function Point

(1) External input (EI)

→ Data from user into system.
eg: - Registration form, Add to Cart.

N-60 Function Point Analysis (FPA)

② External output (EO)

→ Data from system to user
eg:- Confirmation email.

③ External inquiries (EQ)

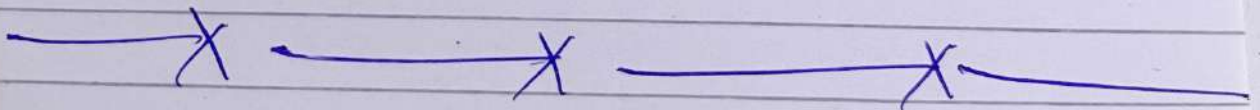
→ request + response, no internal data change.
eg:- Order status.

④ Internal Logical Files (ILF)

→ Data stored inside system & managed by it.
eg:- User data, product catalog.

⑤ External interface files (EIF)

→ Data used by system but stored in external.
eg:- Payment gateway.



V-61 ✓

V-62 Aggregation Vs Composition

Aggregation : A weak relationship b/w (whole & part) where a part can exist independently.

Eg:- Library $\rightarrow$ Book
(If book removed, Library still exist).

Composition : A strong relationship b/w (whole & part) where the part cannot exist independently.

Eg:- House $\rightarrow$ Room
(Room cannot exist without house).

